

ЛАБОРАТОРНАЯ РАБОТА №7

Разработка сценариев на языке оболочки bash

1. Цель работы

1. Изучение принципов разработки сценариев на языке программирования оболочки bash.
2. Разработка собственного сценария с использованием основных синтаксических элементов и команд языка.
3. Знакомство с методикой отладки сценариев для поиска и устранения ошибок.

2. Теоретическая часть

Основы разработки сценариев оболочки

Взаимодействие пользователя с операционной системой осуществляется через оболочку, которая представляет собой внешнюю программу.

Сразу после запуска оболочки производится её инициализация для установки ряда параметров. При этом оболочкой производится чтение двух файлов: */etc/profile* и *~/.profile*. В первом из них содержатся настройки параметров, общие для всех пользователей. Во втором файле каждый пользователь может разместить свои собственные настройки для работы с оболочкой.

Пользовательская оболочка может быть запущена на выполнение в двух режимах – интерактивном и не интерактивном. Когда оболочка выдаёт пользователю приглашение, она работает в интерактивном режиме. Это означает, что оболочка принимает ввод от пользователя и выполняет команды, которые пользователь укажет. Оболочка называется интерактивной, поскольку она взаимодействует с пользователем. Завершение работы с оболочкой в этом случае происходит по команде пользователя.

В не интерактивном режиме оболочка не взаимодействует с пользователем. Вместо этого она читает команды из некоторого файла и выполняет их. Когда будет достигнут конец файла, оболочка завершится. Запуск оболочки в не интерактивном режиме можно осуществить следующим способом:

\$ /bin/sh имя_файла

Здесь *имя_файла* — файл, содержащего команды для выполнения. Такой файл называется сценарием оболочки. Он является текстовым файлом и может быть создан любым доступным текстовым редактором.

Пользователю может представлять неудобство каждый раз указывать имя программы-оболочки */bin/sh* для выполнения сценария. Для того чтобы иметь возможность выполнять сценарий, набирая только его имя, прежде всего необходимо сделать его исполняемым. Для этого необходимо установить соответствующие права доступа к файлу с помощью команды *chmod* следующим способом:

chmod u+x имя_файла

Кроме этого, необходимо явно указать, какая оболочка должна использоваться для выполнения данного сценария. Это можно сделать, разместив в первой строке сценария

последовательность символов `#!/bin/sh` Здесь указывается, что для выполнения сценария необходимо использовать оболочку `/bin/sh`

Сценарий может содержать комментарии. Комментарий — это оператор, который может размещаться в сценарии оболочки, но оболочкой не исполняется. Комментарий начинается с символа `#` и продолжается до конца строки.

Ниже приведён пример короткого сценария:

```
#!/bin/sh
#пример короткого сценария
date
who
```

Составные команды

Кроме простых команд можно формировать составные команды. Они представляют собой упорядоченный набор простых команд, связанных между собой при помощи специальных операторов управления (ОУ). Составная команда имеет следующий синтаксис:

команда ОУ команда ОУ ... ОУ команда

В таблице приведены наиболее часто используемые операторы управления и даны пояснения по выполнению соответствующих составных команд.

Оператор управления в составной команде	Правило выполнения
команда1 ; команда2	«команда2» выполняется после завершения выполнения «команда1 ».
команда1 && команда2	«команда2» выполняется после завершения выполнения «команда1» если код завершения «команда1» 0 (истина).
команда1 команда2	«команда2» выполняется после завершения «команда1» если код завершения «команда1» отличен от нуля (ложь).

Переменные

Переменная это «слово», которому приписано значение. Оболочка разрешает создавать и удалять переменные, а также присваивать им значения. В большинстве случаев разработчик ответственен за управление переменными в сценариях. Использование переменных позволяет создавать гибкие, легко адаптируемые сценарии. Определяются переменные следующим образом (отсутствие пробелов до и после символа «=» существенно):

имя_переменной—значение

Например, `MY_NAME=Sergey` определяет переменную с именем `MY_NAME` и присваивает ей значение `Sergey`. Имена переменных определяются по тем же правилам, что и в языке программирования C. В переменной можно сохранять любое нужное значение.

Особый случай когда это значение содержит пробелы. Для правильного выполнения такого действия указанное значение достаточно заключить в кавычки.

Для того чтобы получить значение переменной, перед её именем необходимо поставить знак \$. В том случае, когда некоторая переменная становится ненужной, её можно удалить операцией *unset* *имя_переменной*. Ниже приведён пример, иллюстрирующий работу с переменными в сценариях.

```
#!/bin/sh
#пример операций над переменными
MY_NAME=Sergey
MY_FULL_NAME="Sergey Borisovich Sidorov"
echo name = $MY_NAME and full name = $MY_FULL_NAME
unset MY_NAME
```

В результате выполнения команды *echo* на терминал будет выдано сообщение:
name=Sergey and full name = Sergey Borisovich Sidorov

Все рассмотренные выше примеры переменных — это примеры скалярных переменных, то есть таких, которые могут хранить только одно значение. Наряду с ними можно использовать *переменные-массивы*. Доступ к элементам массива осуществляется операцией индексирования *[]*. Язык программирования оболочки не требует предварительного объявления переменной массивом с указанием его размерности. При этом отдельные элементы массива создаются по мере доступа к ним. Ниже приведён пример работы с массивом NAME.

```
#!/bin/sh
#пример использования массива
NAME[0]=Sergey
NAME[1]=Katya
NAME[2]=Liza
echo все имена: ${NAME[*]}
echo ${NAME[1]} и ${NAME[2]} сестры
```

Если вместо индекса элемента использовать *, результатом выражения будет список значений всех элементов массива (в данном случае таковых три).

Область видимости переменных

Каждая переменная имеет свою *область видимости*. Допустим, в сценарии определена некоторая переменная. Что будет с ней после завершения этого сценария, доступна ли она из других сценариев, вызываемых исходным? В языке оболочки все переменные делятся на три категории: локальные переменные, переменные окружения и переменные оболочки.

Локальная переменная — это такая переменная, которая существует только внутри конкретного экземпляра оболочки. Она не доступна программам, запускаемым на выполнение из этой оболочки. Все рассматривавшиеся ранее переменные были локальными.

Переменная окружения — это переменная, которая доступна любой программе, запущенной из данной оболочки. Некоторые программы нуждаются в определённых переменных окружения. В таком случае в сценарии их можно определить.

Переменная оболочки – это специальная переменная, которая устанавливается оболочкой и необходима ей для корректной работы. Некоторые из них являются переменными окружения, а некоторые – локальными. В переменных с именами 1, 2, 3 сохраняются параметры командной строки. То есть если некий сценарий *script* запустить в виде *script something*, то в его оболочке *\$1=something*.

Переменную можно разместить в окружении, выполнив команду экспорта: *export имя_переменной*. В результате выполнения следующего сценария на терминал будут выданы значения всех переменных окружения оболочки и, в том числе, переменной MY_NAME.

```
#!/bin/sh
# пример размещения переменной в окружении
MY_NAME=Sergey ; export MY_NAME;
set
```

Ниже в таблице приводится список некоторых стандартных переменных окружения и переменных оболочки с указанием их назначения и примера их инициализации

Имя переменной	Краткое описание
HOME	Содержит путь к домашнему каталогу пользователя
TERM	Тип используемого терминала <i>TERM=vt100</i>
PATH	Определяет список каталогов, в которых оболочка ищет запускаемые на выполнение программы <i>PATH=/bin:/usr/bin</i>
MANPATH	Определяет список каталогов, в которых программа <i>man</i> ищет справочную информацию по запрошенной команде <i>MANPATH=/usr/man:/usr/share/man</i>
PWD	Содержит путь к текущему рабочему каталогу
RANDOM	Формирует случайное целое число из диапазона 0..32767 каждый раз при доступе к этой переменной
?	Результат выполнения предыдущей команды
HOSTNAME	Имя узла (компьютера), на котором выполняется оболочка
SECONDS	Число секунд, прошедших с момента запуска оболочки

Средства ввода-вывода

Задачи ввода—вывода могут быть решены посредством использования специальных команд *echo*, *printf*, *read*. Команда *echo* выдаёт на стандартное устройство вывода значения всех своих параметров. Команда *printf* подобна своему аналогу функции *printf* в программах на языке C. Первым параметром указывается форматная строка, а далее перечисляются выдаваемые значения.

Команда *read имя1 имя2 ... имяN* считывает со стандартного устройства ввода строку, выделяет из неё отдельные слова (группы символов, отделяемые пробелами) и каждое слово заносит в указанные в качестве параметров соответствующие переменные. При этом если переменных меньше, чем выделенных слов, то в последнюю из них будет занесена оставшаяся часть строки.

В качестве примера приведён сценарий, в котором от пользователя запрашивается строка, после чего она выводится на терминал. Параметр *-n* запрещает перевод на следующую строку по окончании вывода командой *echo*, разрешенный по умолчанию.

```
#!/bin/sh
# пример использования средств ввода-вывода
echo -n Введите строку:
read INPUT
printf "%s\n" "$INPUT"
```

Подстановки

Выделяют следующие виды подстановок:

- подстановка имён файлов – позволяет выполнять преобразование строки, содержащей шаблоны, в список имён файлов;
- подстановка переменной – позволяет управлять значением переменной, основываясь на ее состоянии;
- подстановка команды – позволяет захватить вывод команды и подставить его в другую команду
- арифметическая подстановка – позволяет выполнять простые математические вычисления, используя оболочку.

Подстановки имён файлов основывается на использовании различных шаблонов для задания группы файлов в кратком виде вместо полного их перечисления. Шаблон *** отождествляется с любым количеством любых символов в имени файла. Шаблон *?* соответствует одному произвольному символу, а шаблон *[группа символов]* явно определяет набор символов, допустимых в месте его расположения. Ниже приведены примеры использования шаблонов:

```
*.cpp          #список файлов, оканчивающихся на .cpp
module?.h      #список файлов с одним произвольным символом вместо ?
module[aA].cpp #modulea.cpp и ИЛИ moduleA.cpp
```

Механизм подстановки команды состоит в выполнении оболочкой указанного набора команд и последующей подстановки их вывода вместо самих команд. Подстановка команды выполняется при записи команды в обратных апострофах. Например, в результате выполнения команды *DATE='date'* переменная *DATE* примет значение – вывод команды *date*.

Для вычисления значения арифметического выражения *expression* необходимо использовать команду следующего вида: *\$((expression))*. При этом в выражении могут использоваться операции сложения, вычитания, умножения и целочисленного деления, а также круглые скобки для задания порядка вычислений. Например, в результате выполнения следующей команды переменная *foo* примет значение 3.

```
foo=$(((((5+3*2)-4)/2)))
```

Команды ветвления

Язык оболочки содержит два оператора ветвления: *if* и *case*.

Оператор *if* выполняет действия в зависимости от истинности заданного условия и имеет следующий синтаксис:

```
if list1
```

```

then
    commands1
elif list2
then
    commands2
else
    commands3
fi

```

В приведённой записи как *elif*, так и *else* могут отсутствовать. Если тело оператора *if* небольшое, то обычно используют запись в одну строку такую как: *if list1 ; then commands1 ; else commands3 fi*; При этом важно правильно ставить *;*. Алгоритм выполнения оператора *if* следующий:

1. выполняется *list1*
2. если код завершения *list1* 0 (истина), то выполняется *commands1* и оператор *if* завершается;
3. иначе выполняется *list2* и проверяется его код завершения;
4. если *list2* возвратил 0, то выполняется *commands2* и оператор завершается;
5. если *list2* возвратил не нулевое значение, то выполняется *commands3*.

В качестве условий *list1* и *list2* могут использоваться обычные команды, однако наиболее часто – это вызов одной или нескольких *test* команд в виде *test выражение* или (более кратко) [*выражение*]. Подробно команда *test* рассмотрена в следующем разделе. Ниже приведён пример использования оператора *if*:

```

#!/bin/sh
# выполнение программы с контролем её существования
if [ -x $HOME/script ] ; then $HOME/script ; fi ;

```

В данном примере проверяется, существует ли в домашнем каталоге пользователя файл с именем *script* и является ли он исполняемым (атрибут 'x'). Если это так, то сценарий запускает *script* на выполнение.

Если необходимо сделать выбор из многих альтернатив, то более удобно использовать оператор *case*, имеющий следующий синтаксис:

```

case значение in
    шаблон1) список1 ;;
    шаблон2) список2 ;;
    ...
esac

```

Оператор *case* является аналогом оператора *switch* в языке C и работает похожим образом. Строка *значение* сравнивается с каждым из указанных *шаблонов* до тех пор, пока не будет обнаружено соответствие. После обнаружения соответствия выполняется *список*, указанный после шаблона. Два символа *;;* после списка имеют тот же смысл, что и оператор *break* в программах на языке C. Если никаких соответствий не обнаружено, то оператор *case* завершается без выполнения каких-либо действий.

Ниже приведён пример использования оператора *case*. Сценарий ожидает в качестве параметра название фрукта и выдаёт его описание. Если при вызове параметр не указан, то выдаётся диагностическое сообщение.

```

#!/bin/sh

```

```
# печать информации о фруктах
if [ -z "$1" ]
then
    echo "Ошибка: пропущен параметр"
    exit 1
fi
case "$1" in
    apple) echo "Яблоки очень полезны" ;;
    banana) echo "Бананы очень вкусны" ;;
    *) echo "Ничего не могу сказать про " ;1 ;;
esac
```

Синтаксис команды test

Команда *test* *выражение* проверяет указанное *выражение* и заканчивает свою работу с кодом завершения 0 (истина) или (ложь). В качестве выражение могут использоваться выражения 3 типов:

- проверки характеристик файлов;
- сравнения строк;
- числовые сравнения.

Основной синтаксис для проверки характеристик файлов следующий: *test опция имя_файла*. В приведённой ниже таблице сведены возможные опции команды *test* для проверки характеристик файлов.

Опция	Описание
-b	Истина, если файл существует и является блочным специальным файлом.
-c	Истина, если файл существует и является символьным специальным файлом.
-d	Истина, если файл существует и является каталогом.
-e	Истина, если файл существует.
-f	Истина, если файл существует и является обычным файлом.
-h	Истина, если файл существует и является символической ссылкой.
-r	Истина, если файл существует и доступен на чтение.
-s	Истина, если файл существует и имеет ненулевой размер.
-S	Истина, если файл существует и является сокетом.
-w	Истина, если файл существует и доступен по записи.
-x	Истина, если файл существует и доступен на выполнение.

В приведённой ниже таблице сведены возможные способы указания использования команды *test* для проверки строк.

Опция	Описание
-z строка	Истина, если строка имеет нулевую длину.
-n строка	Истина, если строка имеет ненулевую длину.
строка1 = строка2	Истина, если строки совпадают.
строка1 != строка2	Истина, если строки различны.

При использовании числовых сравнений команда *test* имеет следующий синтаксис:
test число1 оператор число2 В приведённой ниже таблице сведены возможные значения операторов при числовых сравнениях.

Опция	Описание
<i>-eq</i>	Истина, если числа равны.
<i>-ne</i>	Истина, если числа не равны.
<i>-lt</i>	Истина, если меньше.
<i>-le</i>	Истина, если меньше или равны.
<i>-gt</i>	Истина, если больше.
<i>-ge</i>	Истина, если больше или равны.

Организация циклов

Язык оболочки позволяет организовывать циклическое выполнение команд. В распоряжение пользователю предлагается несколько вариантов циклов:

- *while*
- *for*
- *select*

Оператор цикла *while* имеет следующий синтаксис:

```
while команда
do
    список
done
```

При выполнении цикла сначала выполняется *команда*. Если результат ненулевой, то происходит выход из цикла, в противном случае выполняется тело цикла *список* и происходит переход на следующую итерацию. Хотя в качестве условия *команда* может использоваться любая допустимая в GNU/Linux команда, обычно это команда *test*. Ниже приведён пример выдачи на терминал десяти последовательных чисел от 0 до 9.

```
#!/bin/sh
# арифметические вычисления
x=0
while [ $x -lt 10 ] # значение переменной x меньше 10 ?
do
    echo $x
    x='expr $x + 1' # увеличение x на 1
done
```

Цикл *for* выполняет команды из *список* для каждого элемента из списка и имеет следующий синтаксис:

```
for имя in элемент1 элемент2 ... элементN
do
    список
done
```

В качестве элементов можно использовать различные шаблоны. В приведённом ниже примере, все файлы из домашнего каталога пользователя, которые заканчиваются на *.bash*, копируются в каталог *scripts* и делаются доступными по чтению всем пользователям.

```
#!/bin/sh
# выполнение операций над группой файлов
for FILE in $HOME/*.bash
do
    cp $FILE ${HOME}/scripts
    chmod a+r ${HOME}/scripts/${FILE}
done
```

Язык программирования оболочки содержит операторы, нарушающие нормальное выполнение цикла. Эти операторы имеют названия – *break* и *continue*. Они работают в точности так же, как и их аналоги в языке C.

Цикл *select* позволяет создавать удобные меню. Он полезен, когда необходимо, чтобы пользователь выбрал один элемент из предлагаемого списка. Оператор *select* имеет такой же вид, как и оператор *for*, за исключением ключевого слова.

При выполнении данного оператора цикла все элементы из списка высвечиваются на экране вместе с их порядковыми номерами, после чего появляется специальное приглашение для ввода. Обычно оно имеет вид *#?*. Введённый пользователем номер пункта меню записывается в переменную *REPLY*. Если *\$REPLY* содержит номер пункта меню, то в переменную *имя* заносится значение соответствующего элемента из списка. В противном случае список будет выдан заново.

После того как пользователем будет сделан допустимый выбор, выполняться команды из *список*, после чего выполнение цикла повторяется с самого начала (высветится приглашение для ввода и т. д.). Для повторного отображения меню в ответ на приглашение ввода следует нажать клавишу *Enter*. Выход из цикла осуществляется теми же средствами, что и для *while* и *for*.

Ниже приводится пример использования оператора *select*. У пользователя запрашивается тип устройства «мышь», и в зависимости от сделанного выбора выполняются определённые действия. В рассматриваемом случае это просто выдача сообщения, подтверждающая сделанный выбор. В приведённом сценарии обратим внимание на вторую строчку. В большинстве случаев действующий по умолчанию вид приглашения не устраивает. Определяя переменную *PS3*, можно задать необходимый текст приглашения.

```
#!/bin/sh
# “конфигурирование” устройства «мышь»
PS3="Выберите тип устройства «мышь»:"
```

```

select ITEM in Microsoft Logitech ps2 none
do
    case $ITEM in
        Microsoft) echo "действия по «мышь» Microsoft " ;;
        Logitech) echo "действия по «мышь» Logitech " ;;
        ps2) echo "действия по «мышь» ps2 " ;;
        none) echo "выбрано – нет «мышь» " ;;
    esac
break
done

```

Использование функций

В сценариях оболочки допустимо определять и использовать функции. Под функцией понимается поименованная группа команд. Во всех случаях, когда в сценарии присутствует повторяющийся код с возможно небольшими вариациями, необходимо рассмотреть возможность применения функции. Так же логически завершённую группу команд удобнее представить отдельной функцией, что позволяет упростить чтение и восприятие сценария. Определение функции имеет следующий синтаксис:

```

имя_функции ()
{
    список команд
}

```

Определение функции обязательно должно предшествовать её первому использованию. В отличие от языка C++ в языке оболочки *bash* отсутствуют средства предварительного объявления функции (аналоги прототипов функций). Вызов функции осуществляется путём указания её имени в качестве команды.

Ниже приведён пример сценария, в котором определяется функция *about*, предназначенная для выдачи информации о сценарии, и иллюстрируется её использование:

```

#!/bin/sh
# определение функции
about()
{
    echo пример сценария с функцией
    echo (C) Иванов И.И., 2021
}
about          # вызов функции
# ... продолжение сценария

```

Функции при вызове можно передавать аргументы и при своём завершении функция может возвращать результат выполнения. Функция ссылается к переданным параметрам через позиционные переменные - \$1, \$2 и так далее. Результат работы возвращается с

помощью оператора *return*. Ниже приведён пример сценария, в котором иллюстрируется доступ функции к переданным в неё параметрам и вызов функции с передачей параметров.

```
#!/bin/sh
# функция с параметром
double_echo()
{
    if [ -n "$1" ]
    then
        echo $1
        echo $1
    fi
    return 0
}
double_echo параметр           # вызов функции с одним параметром
double_echo                   # вызов функции без параметров
# ... продолжение сценария
```

3. Практическая часть

Пример 1: Создать скрипт, который выводит содержимое текущего каталога:

```
#!/bin/sh
#пример1
ls -l
```

Пример 2: Создать скрипт, который выводит столицу заданной страны:

```
#!/bin/sh
#пример2
if [ -z "$1" ]
then
    echo "Ошибка: пропущен параметр"
    exit 1
fi
case "$1" in
    Россия) echo "Москва" ;;
    Иран) echo "Тегеран" ;;
    *) echo "Не знаю" ;;
esac
```

Пример 3: Вывести на экран все числа от 0 до 100:

```
#!/bin/sh
#пример3
x=0
while [ $x -lt 100 ]
do
    echo $x
    x=$(( $x + 1 ))
done
```

Пример 4: Создать скрипт, с помощью которого все файлы из домашнего каталога пользователя, которые заканчиваются на .sh, копируются в каталог scripts и делаются доступными по чтению всем пользователям:

```
#!/bin/sh
#пример4
for FILE in $HOME/*.sh
do
    cp $FILE ${HOME}/scripts
    chmod a+r ${HOME}/scripts/${FILE}
done
```

```
done
```

Пример 5: Создать скрипт, который выводит количество дней в выбранном месяце и запускает его с помощью bash:

```
#!/bin/sh
#пример5
#echo = "Выберите месяц:"
select M in Январь Февраль Март Апрель Май Июнь Июль Август Сентябрь
Октябрь Ноябрь Декабрь
do
    case $M in
        Январь) echo 31 ;;
        Февраль) echo 28 ;;
        Март) echo 31 ;;
        Апрель) echo 30 ;;
        Май) echo 31 ;;
        Июнь) echo 30 ;;
        Июль) echo 31 ;;
        Август) echo 31 ;;
        Сентябрь) echo 30 ;;
        Октябрь) echo 31 ;;
        Ноябрь) echo 30 ;;
        Декабрь) echo 31 ;;
    esac
    break
done
```

```
$bash primer5
```

Пример 6: Создать функцию, которая выводит вашу фамилию при вызове:

```
#!/bin/sh
#пример6

my_surname()
{
    echo Мамедли
}

my_surname
```

Пример 7: Создать функцию, которая выводит сумму двух чисел:

```
#!/bin/sh
```

#пример7

```
my_sum()  
{  
    s=0  
    if [ -n "$1" ]  
    then  
        s=$1  
    fi  
    if [ -n "$2" ]  
    then  
        s=$(( $s + $2 ))  
    fi  
    echo $s  
}
```

my_sum 3 5

4. Задание

При старте сценарий необходимо выдавать информацию об авторе в виде:
(C) имя фамилия группа

1. Найти длину окружности L и площадь круга S заданного радиуса R .
2. Даны два числа a и b . Найти их среднее арифметическое.
3. Даны два ненулевых числа. Найти сумму, разность, произведение и частное их модулей.
4. Даны целые числа a , b , c , являющиеся сторонами некоторого треугольника. Проверить, является ли треугольник со сторонами a , b , c равносторонним.
5. Даны координаты двух различных полей шахматной доски x_1, y_1, x_2, y_2 (целые числа, лежащие в диапазоне 1–8). Проверить, может ли слон за один ход перейти с одного поля на другое.
6. Даны два целых числа A и B ($A < B$). Вывести в порядке возрастания все целые числа, расположенные между A и B (включая сами числа A и B), а также количество N этих чисел.
7. Дано вещественное число – цена 1 кг конфет. Вывести стоимость 1, 2, ..., 10 кг конфет.
8. Дано целое число N ($N > 0$), являющееся некоторой степенью числа 2: $N = 2^K$. Найти целое число K — показатель этой степени.
9. Два двузначных числа, записанных одно за другим, образуют четырёхзначное число, которое делится на их произведение. Найти эти числа.
10. В восточном календаре принят 60-летний цикл, состоящий из 12-летних подциклов, обозначаемых названиями цвета: зеленый, красный, желтый, белый и черный. В каждом подцикле годы носят названия животных: крысы, коровы, тигра, зайца, дракона, змеи, лошади, овцы, обезьяны, курицы, собаки и свиньи. По номеру года определить его название, если 1984 год — начало цикла: «год зеленой крысы».